

Context-Aware Linguistic Steganography: A Generative Workflow using Large Language Models

NASOUH ALOLABI, Higher Institute for Applied Sciences and Technology, Syria

RIAD SONBOL, Higher Institute for Applied Sciences and Technology, Syria

This paper proposes a context-aware **black-box** linguistic steganography workflow for threaded discussions that carries the payload through a *selection channel* rather than token-level manipulation. The sender reconstructs a shared context corpus, selects both a reply target and a semantic angle, and uses a stochastic LLM to realize the choice as an ordinary visible reply. The receiver rebuilds the same context, recovers the chosen selections, and decodes the payload from the published comment. The method is designed to remain natural in conversation, avoid visible control markers, and support recovery using only shared assumptions and the agreed temporal window. We evaluate the approach using naturalness and distributional divergence metrics on successful runs, and report generation and decode failures separately.

Additional Key Words and Phrases: Linguistic Steganography, Large Language Models, LLMs, Natural Language Processing, NLP, Black-box Steganography, Semantic Embedding, Agentic Workflows, Imperceptibility

PREPRINT NOTE

This manuscript is a preprint dated May 23, 2026.

1 INTRODUCTION

Linguistic steganography focuses on the covert embedding of secret information within ostensibly innocuous textual carriers. Traditional methodologies, primarily predicated on lexical substitution and synonym manipulation [1], often struggle with the inherent low redundancy of natural language. Consequently, even minor statistical deviations can provide sufficient signal for modern automated detection systems [2–4]. The emergence of Large Language Models (LLMs) has catalyzed a paradigm shift, transitioning the field from text modification to de novo text generation [5–7]. By sampling directly from a model’s learned probability distribution, these generative approaches theoretically achieve high levels of statistical imperceptibility.

However, existing LLM-based steganographic systems remain constrained by a fundamental trade-off between statistical security and perceptual naturalness [5–7]. While a model may output grammatically and statistically fluent sentences, these often lack the deep semantic coherence required for specific communicative contexts, such as threaded social media discussions [8–10]. A comment that is technically flawless yet pragmatically irrelevant to the ongoing discourse serves as a conspicuous anomaly under human or semantic-level scrutiny.

To address these challenges, we introduce a context-aware steganographic framework that elevates the embedding domain from the lexical level (individual tokens) to the semantic level (conversational decisions). Instead of manipulating token distributions, our scheme carries the payload through a *selection channel*: bits map to (i) the index of the reply target within a deterministically flattened comment tree, and (ii) the index of a context-conditioned *Angle* within a shared angle set that both parties reconstruct deterministically from the public thread and reproducible research. A language model then generates an ordinary visible reply that serves as a *decodable witness* of the selected Angle; we do not rely on invisible characters or control Unicode. A sender-side verification loop closes the system by simulating

Authors’ addresses: Nasouh Alolabi, Higher Institute for Applied Sciences and Technology, Damascus, Syria; Riad Sonbol, Higher Institute for Applied Sciences and Technology, Damascus, Syria.

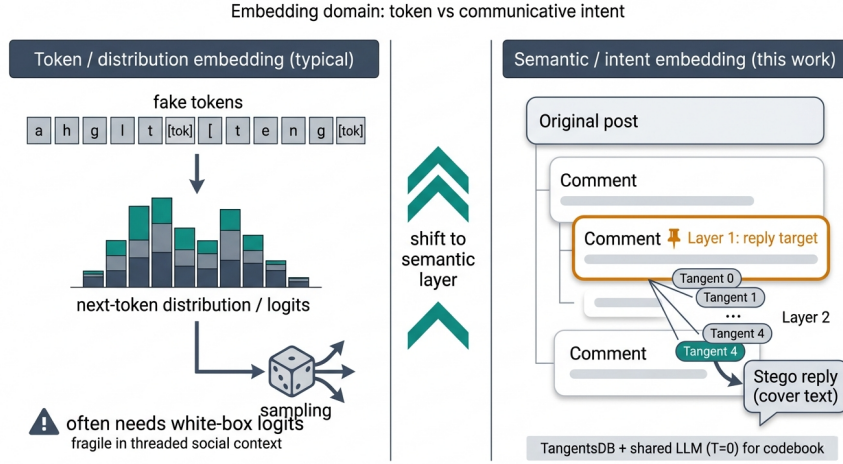


Fig. 1. Conceptual contrast between token-level embedding and our two-layer semantic embedding over a threaded discussion.

receiver decoding before publication. Figure 1 contrasts token-level embedding with our two-layer selection-channel embedding over a threaded discussion.

The primary contributions of this paper are:

- A two-layer, selection-channel embedding architecture for threaded discussions that encodes bits into reply targeting and Angle selection, with visible text serving as a decodable witness rather than a hidden carrier.
- A deterministic reconstruction pipeline (thread + reproducible research) that enables sender/receiver parity without white-box model access or side-channel metadata exchange.
- A closed-loop verification mechanism that simulates decoding pre-publication and retries synthesis to improve recoverability under stochastic generation.

2 BACKGROUND

2.1 Linguistics-Based Steganography

Steganography is distinguished from cryptography by its primary objective: the achievement of imperceptibility rather than mere confidentiality. The fundamental goal is to obscure the very existence of a communication channel. This is traditionally conceptualized through the "Prisoners' Problem" [11], wherein Alice must transmit a message to Bob under the scrutiny of an adversarial warden. Should the warden identify any deviation from expected behavior, the communication channel is terminated. Figure 2 illustrates this monitored-channel abstraction.

2.2 Generative Steganography in the Era of LLMs

Traditional textual steganography involves the modification of a static cover text, a process that frequently introduces detectable statistical or linguistic artifacts. In contrast, generative steganography [7] utilizes probabilistic models to synthesize the cover text de novo.

- **Syntactic/Token-Level Embedding:** Prevailing methodologies often embed information by manipulating the sampling distribution of the subsequent token (e.g., through vocabulary partitioning or binning). However, these techniques typically necessitate "white-box" access to the model's internal logits.

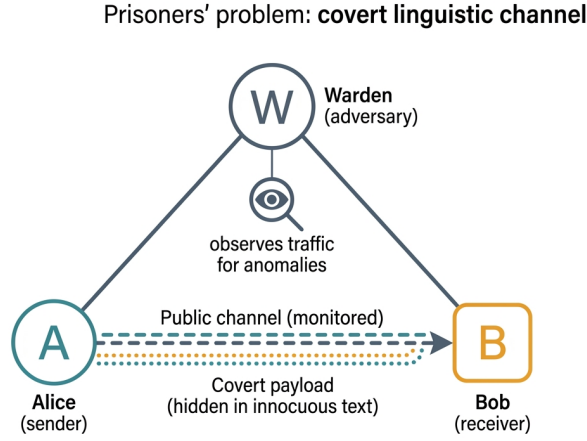


Fig. 2. The prisoners' problem: Alice must reach Bob over a channel monitored by an adversary (warden); steganography hides the existence of the covert payload.

- **Black-Box Generative Architectures:** Emerging frameworks interact with language models solely through API-driven prompts. Our research operates within this paradigm, leveraging the instruction-following capabilities of LLMs rather than direct manipulation of internal weight distributions.

Threat model. We assume a passive observer (human or automated) can read the public thread and the published reply, but does not possess the shared operational parameters used by the sender and receiver (deterministic preprocessing and search constraints, deterministic model settings for reconstruction, and the agreed temporal synchronization window). The platform itself is treated as an untrusted carrier that observes only public content.

Linguistic steganography specifically refers to the concealment of information within natural language through the generation or modification of text while maintaining semantic and syntactic fluency [12]. While LLM-based systems have significantly improved the quality of generated covers, they remain susceptible to the fundamental tension between statistical imperceptibility and contextual naturalness [2, 13].

2.3 The Semantic Context Challenge

A critical limitation of existing generative systems is their tendency to produce content in isolation or based on abstract prompts. In the context of real-world communicative environments, such as social media, "naturalness" is inextricably linked to conversational relevance. Consequently, a sophisticated adversary can identify steganographic content not by identifying syntactic errors, but by analyzing its pragmatic misalignment with the surrounding discourse [14]. The framework proposed in this research specifically addresses this semantic-level vulnerability.

3 METHODOLOGY

This paper proposes a black-box steganographic technique that embeds information within the semantic architecture of a conversation. The methodology is predicated on a standard covert communication scenario involving Alice (the sender) and Bob (the receiver), communicating over a public social media platform. While this study utilizes Reddit as a primary case study, the underlying principles are platform-agnostic and transferable to any threaded discussion environment. Figure 3 sketches the end-to-end covert scenario and shared synchronization assumptions.

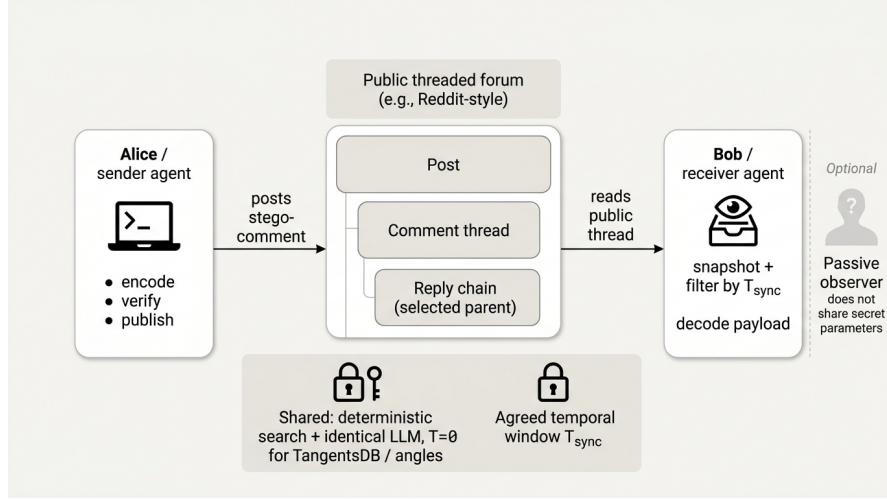


Fig. 3. Covert communication scenario: Alice and Bob use a public threaded channel; synchronization relies on shared operational parameters and a negotiated temporal window.

3.1 System Architecture

The core of the proposed framework centers on a multi-layer embedding process that leverages both the *spatial location* of the response and the *thematic subject* of the content as carriers for the secret payload. By mapping binary data to high-level conversational actions—specifically, the selection of a reply target and the selection of an *Angle*—the system carries the payload through a selection channel while preserving perceptual naturalness in the visible text. In the implemented workflow, the receiver reconstructs the same pre-sender context (thread snapshot + deterministic research + regenerated angles) and decodes the same selection indices to recover the payload. Figure 7 depicts the two embedding layers and their interaction with synthesis.

3.2 Operational Requirements

To achieve synchronization, both the sender and receiver must maintain access to:

- **Public Communication Channel:** Accounts on the target social media platform.
- **Shared Operational Parameters:** A deterministic search algorithm, post metadata, and predefined search constraints.
- **Deterministic Reconstruction Settings:** A synchronized model/backend configuration for deterministic stages (e.g., temperature=0) so both parties regenerate the same search terms and angle set from the same inputs.
- **Agreed Temporal Window:** A negotiated T_{sync} used to filter thread state so the receiver reconstructs the same parent-comment context seen by the sender.

3.3 Search Determinism & Synchronization Framework

Ensuring that both Alice and Bob operate on the same researched corpus is the cornerstone of reliable decoding. The implemented research stage is a reproducible preprocessing chain rather than an ad hoc web search. A search-term generator first extracts 12–20 unique queries from the post title, body, and URL using the workflow LLM at

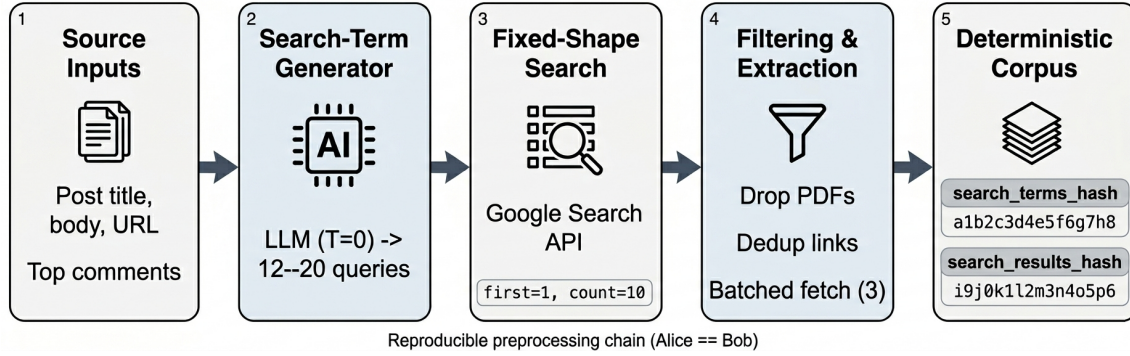


Fig. 4. Deterministic research stage: shared term generation, fixed search parameters, filtering, batched fetching, and integrity hashes for replayable corpora.

temperature=0. Each query is then executed through the Google Custom Search backend with a fixed request shape (first=1, count=10), duplicate links and PDF results are removed, and the surviving URLs are fetched in batches of three. Content extraction uses a crawl4ai-based adapter that normalizes URLs and extracts the primary text content. The resulting flat list of fetched page texts is stored as a search-results corpus, and the pipeline records term and corpus integrity hashes so the same corpus can be replayed by downstream stages. Figure 4 summarizes this deterministic research chain.

3.4 State Synchronization (Temporal Window)

To ensure that both Alice and Bob visualize an identical state of the conversation tree, they must adhere to a negotiated *Temporal Window Duration* (T_{sync}). This synchronization mechanism is operationalized to address the processing latency inherent in the sender's agent:

- (1) **Sender Side:** Alice captures a snapshot of the discussion thread at time t_{start} , performs the multi-layer embedding, and subsequently publishes the generated stego-comment at $t_{publish}$.
- (2) **Receiver Side:** Upon identifying the stego-contribution published at $t_{publish}$, Bob filters the thread's historical state to include only those comments established prior to $t_{publish} - T_{sync}$.

The agreed-upon duration T_{sync} ensures that the "Comment Chain" retrieved by the receiver precisely replicates the context available to the sender at t_{start} , even if new comments were published during the sender's computational cycle. This prevents state divergence and ensures that the decoder can confidently identify the exact parent comment and conversational state used for encoding. Figure 5 shows the temporal window relative to t_{start} and $t_{publish}$.

3.5 Payload Optimization & Compression

Prior to embedding, the raw payload undergoes a transformation layer designed to minimize bitstream length. We implement a *dynamic dictionary-based compression* scheme that uses dynamic programming to minimize the total bit-cost:

- (1) **Dictionary Synthesis:** A session-specific linguistic dictionary is synthesized from the post body, the flattened comment bodies, and the fetched search-results text corpus.
- (2) **Optimal Path Selection:** The compression engine scans payload substrings up to 250 characters and evaluates two encoding modes for each segment:

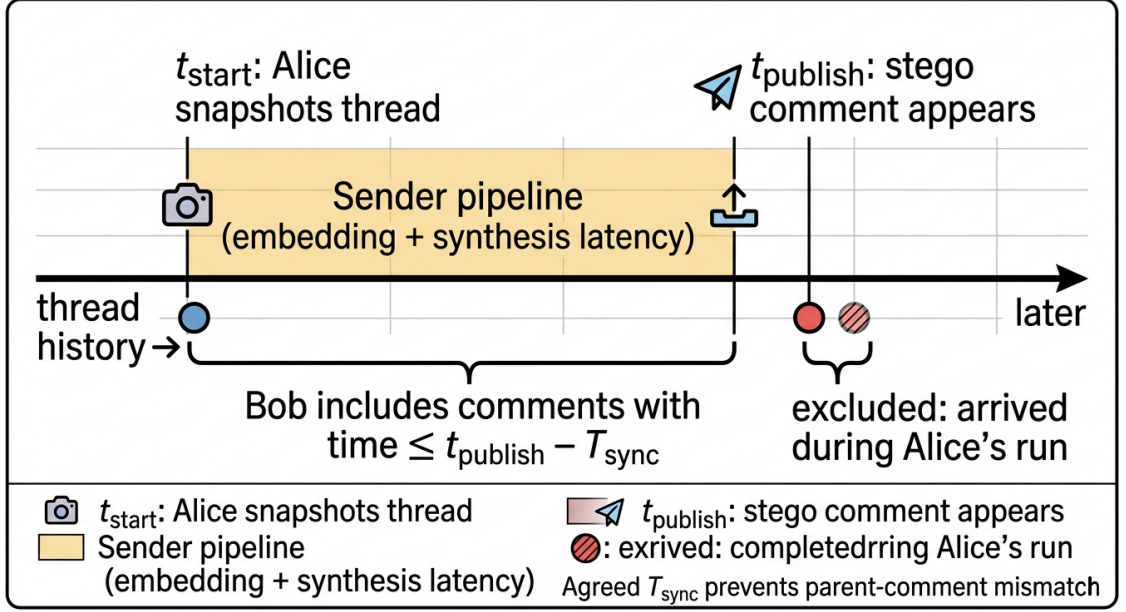


Fig. 5. Temporal synchronization: Bob's reconstructed thread state matches Alice's by agreeing on T_{sync} and filtering comments relative to t_{publish} .

- *Literal Encoding*: Encodes the raw UTF-8 bitstream with a short length header.
 - *Dictionary Encoding*: Encodes a reference to the dynamic dictionary using a tuple of (*Dictionary Index*, *Offset*, *Length*).
- (3) **Bit-Cost Optimization**: The dynamic program selects the sequence of encoding modes that minimizes the total compressed length. If the dictionary path is not shorter than standard UTF-8, the compressor falls back to the standard representation.

Figure 6 outlines dictionary construction and the literal versus dictionary encoding modes.

3.6 Two-Layer Semantic Embedding Architecture

Figure 7 provides the schematic overview of how the two embedding layers split the payload before synthesis.

3.6.1 Layer 1: Spatial Embedding (Reply Targeting). The primary embedding layer determines the *architectural placement* of the steganographic contribution.

- (1) The system flattens the nested comment tree with a deterministic depth-first walk.
- (2) The bit-width for this layer is dynamically calculated as $w_1 = \lceil \log_2(n + 1) \rceil$, where n is the number of available comments.
- (3) Index 0 corresponds to a top-level reply to the post, while any non-zero index selects a comment whose parent chain is reconstructed through parent-identifier and link-identifier lookups.

3.6.2 Layer 2: Intent-Based Embedding (Angle Selection). The secondary layer determines the *communicative intent*.

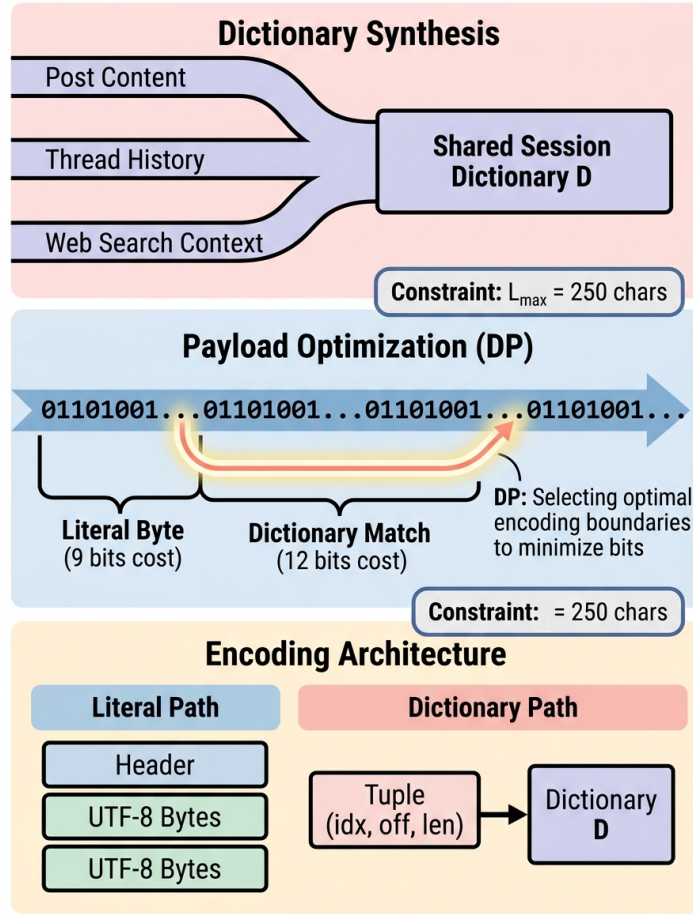


Fig. 6. Payload compression builds a session dictionary from thread and research text, then chooses literal versus dictionary-encoded segments to minimize total bit cost.

- (1) **Angle Set Reconstruction:** Using the deterministic preprocessing chain (Section 3.3) and the filtered thread snapshot, both parties regenerate the same ordered list of candidate Angles, each described by a category label, a source quote, and a tangent field (the thematic direction).
- (2) **Dynamic Bit-Mapping:** The bit-width for this layer is computed from the total number of Angles, and the flattened list is assigned sequential idx values.
- (3) **Angle Selection:** The corresponding payload bits are mapped to a specific Angle index. During synthesis, the selected Angle is paired with the reply context and relevant research excerpts to produce a plausible cover reply.

3.7 Generative Synthesis & Verification

Following the selection of the target context and selected Angle, the sender initiates the cover text synthesis process.

- **Synthesis:** A configurable generative language model (run at non-zero temperature) produces a small fixed set of short, thread-appropriate reply candidates as a JSON list of non-empty strings. The prompt conditions on post metadata, the selected comment chain, the selected Angle (tangent), and relevant research excerpts.

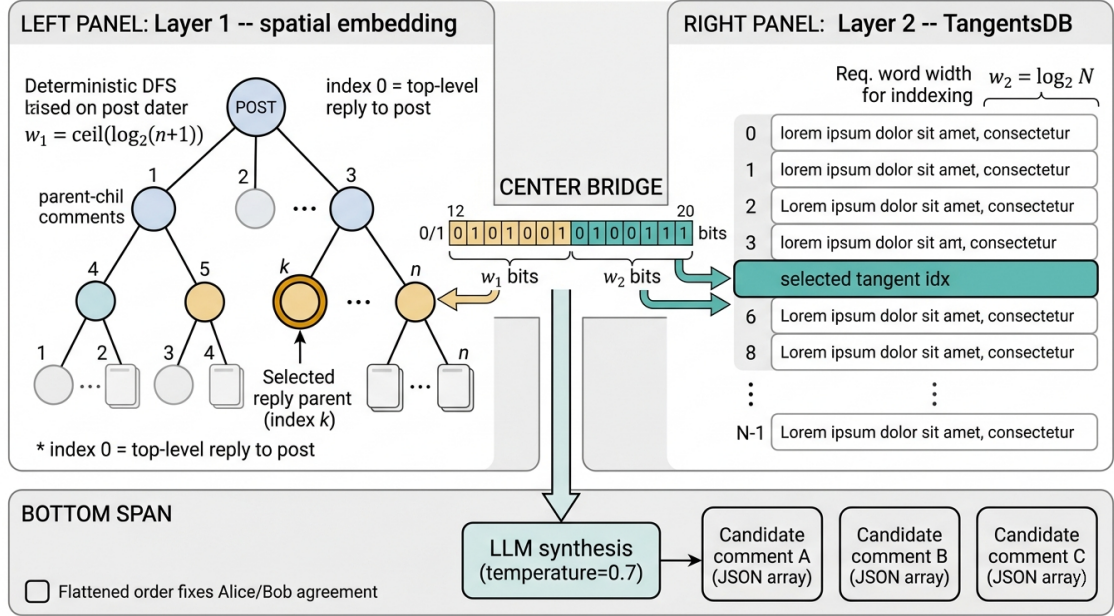


Fig. 7. Two-layer embedding: Layer 1 selects reply placement in a deterministically flattened thread; Layer 2 selects an Angle from a shared, deterministically reconstructed angle set; combined bits drive generative synthesis.

- **Verification Loop:** Before the stego-text is committed to the public channel, the sender executes a local *Decoding Simulation* to verify that the generated text remains recoverable.
- **Recursive Refinement:** If the simulation fails to recover the intended Angle, the sender re-executes synthesis without changing the embedding parameters. The non-zero temperature ensures output variation; the retry loop is capped by a fixed attempt budget.

Figure 8 summarizes sender-side verification and receiver-side recovery.

3.8 Decoding & Intent Recovery

To recover the hidden intent (Layer 2), the receiver first reconstructs the pre-sender context deterministically, then decodes the Angle selection from the visible reply:

- (1) **Context Reconstruction:** The receiver locates the candidate stego comment, removes it from the thread, applies the temporal cutoff ($t_{\text{publish}} - T_{\text{sync}}$), and reruns deterministic preprocessing (URL normalization, research, and angle generation) to reproduce the same ordered Angle list.
- (2) **Semantic Filtering:** A semantic similarity model shortlists the top- N Angles whose text descriptions best match the observed stego reply.
- (3) **Discriminative Identification:** A deterministic identification prompt is executed on the shortlisted candidates to return the most likely Angle index:

"Identify the specific Angle from the provided candidates... return the index only."

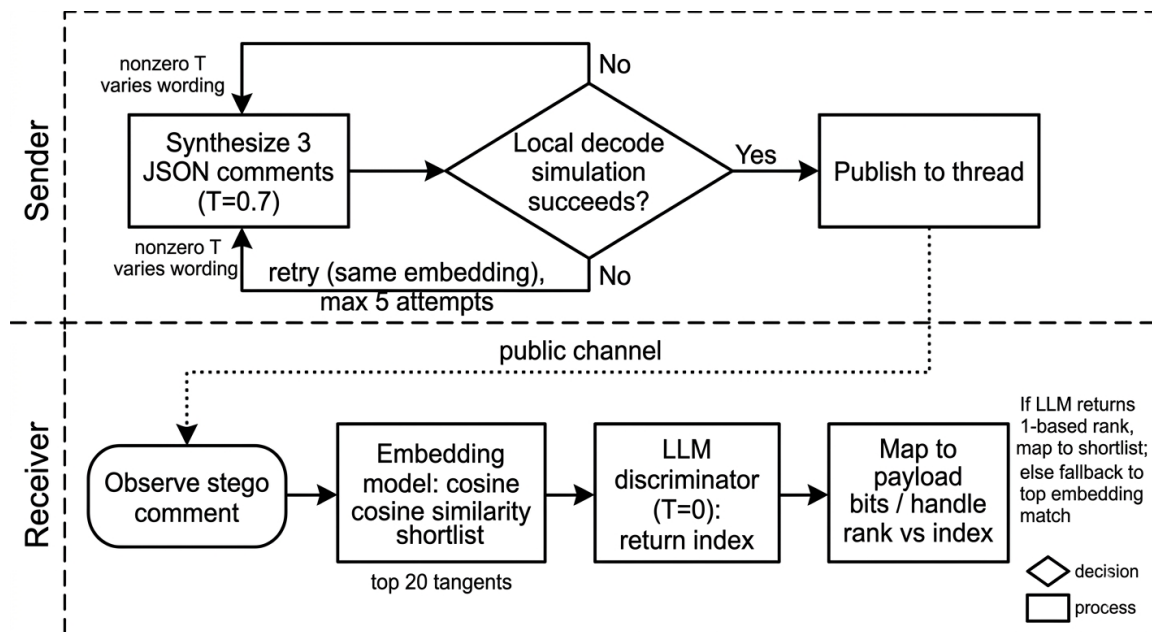


Fig. 8. Sender-side verification retries synthesis until decode succeeds; the receiver recovers Layer 2 via semantic shortlisting and a discriminative identification step.

If the discriminator returns a 1-based rank rather than a raw index, the pipeline maps the rank back to the shortlist; otherwise it falls back to the top semantic match. Combined with Layer 1 (reply targeting), the recovered indices reconstruct the same selection bits and allow recovery of the compressed payload.

4 IMPLEMENTATION

The practical implementation follows the same stage order as the workflow runner: data load, research, angle generation, stego synthesis, and decode. Each stage persists artifacts to disk so later stages can reuse the exact same corpus and hashes, which keeps the sender and receiver aligned across retries.

4.1 Architectural Components

The system is operationalized through a series of deterministic pipeline stages. Core components include:

- **Data load stage:** Fetches unresolved URL content and replaces the post body with the extracted text.
- **Research stage:** Generates search terms from the post title, text, and URL using a deterministic LLM configuration (e.g., temperature=0), queries a fixed-shape search backend, removes duplicate links and PDFs, and fetches page text in bounded batches. The stage persists a flat search-results corpus together with reproducibility hashes.
- **Angle generation stage:** Flattens the corpus assembled from the post text, search-results corpus, and comments, then calls the angle runner to produce triplets consisting of a category, a source quote, and a tangent. Long inputs are split into 30k-character segments and batched up to 150k characters. If the backend path fails, the pipeline falls back to a direct LLM prompt that uses the same JSON schema.
- **Stego and decode stages:** Combine the compression layer, comment targeting, semantic shortlist, and verification loop. The encoder generates a small fixed number of candidate comments for the selected Angle, decodes them immediately, and retries synthesis if the selected Angle is not recoverable.

Figure 9 aligns these components with the staged implementation and on-disk artifacts.

4.2 Model Configuration

The implementation uses different model roles for different stages, but the exact model IDs are treated as configurable parameters:

- **Deterministic stages (term/angle generation):** Run at temperature=0 to ensure reproducible reconstruction from the same inputs.
- **Stego synthesis:** Runs at non-zero temperature to generate diverse candidate replies for a fixed embedding choice.
- **Semantic matching and decode:** Uses a semantic similarity model for shortlist ranking, followed by a deterministic discriminative identification prompt (e.g., temperature=0) to select the final Angle index.

5 EVALUATION

This section reports an artifact-based assessment of the workflow, focusing on (i) embedding capacity of the selection channel, (ii) naturalness and distributional divergence of successful stego samples, and (iii) operational reliability under the sender-side verification loop. We center results on a single clean end-to-end run artifact set and explicitly separate generation/decode failures from infrastructure errors.

5.1 Capacity Analysis

The channel capacity of Layer 2 is fundamentally dependent on the size of the shared, deterministically reconstructed angle set. If the reconstructed set contains N Angles, then Layer 2 contributes $\log_2 N$ bits per message instance (rounded

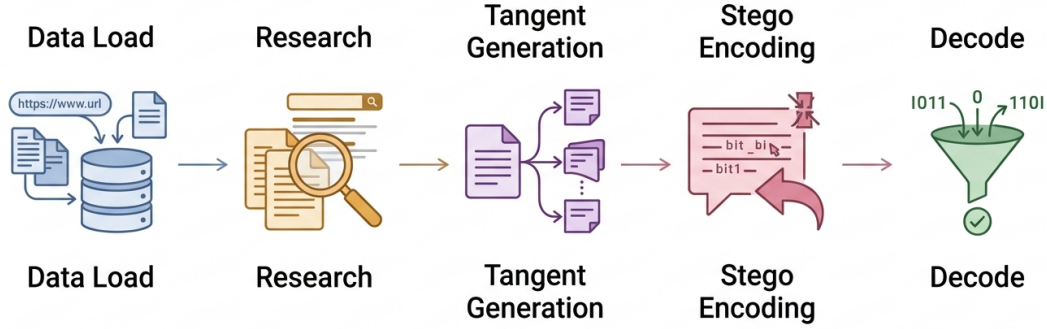


Fig. 9. Implementation stages and persisted artifacts align with the methodology: each stage is deterministic and replayable where required for sender–receiver parity.

Table 1. Perplexity and distribution divergence for stego comments (GPT-2 PPL; KL/JSD on word unigrams).

Category	Metric	Reference set (this run)	Mean	Range (min / max)
Perplexity	PPL (GPT-2)	183 successful stego strings	≈ 142.3	22.8 – 653.8
Divergence	KL	Matched primary post	≈ 9.24	—
Divergence	KL	Global comment corpus	≈ 6.50	—
Divergence	JSD	Matched primary post	≈ 0.590	—
Divergence	JSD	Global comment corpus	≈ 0.588	—

up to an integer bit-width in the implementation). In practice, broader threads and richer research corpora tend to yield larger and more diverse angle sets than narrowly constrained topics, increasing available capacity.

5.2 Perceptual & Statistical Imperceptibility

Imperceptibility is evaluated through the dual lenses of semantic coherence and contextual integration. Unlike traditional steganographic methodologies that risk producing disjointed or statistically anomalous text, our intent-based approach aims to ensure that each stego contribution maintains a clear thematic alignment with the surrounding discourse. In this work we report GPT-2 perplexity and word-unigram KL/JSD as coarse statistical proxies, noting that these metrics do not fully capture long-context semantic quality [15] and do not replace dedicated steganalysis benchmarks.

Table 1 summarizes perplexity (GPT-2) and word-unigram divergence against two within-run reference distributions: the matched primary post and a global comment corpus. Values are computed over the successful, receiver-validated stego samples from a single clean end-to-end run (requested=200, succeeded=183).

Definitions. Perplexity is computed from average per-token log-probability, $2^{-\frac{1}{n} \sum_{i=1}^n \log_2 P(x_i | x_{<i})}$: lower PPL indicates more natural continuations under the model; higher PPL indicates surprising or awkward text. The observed maximum (~ 654) likely reflects a small number of pathological samples that are highly unnatural to GPT-2.

Kullback–Leibler (KL) divergence compares stego word unigrams P_S to a baseline P_C via $\sum_w p_{\text{stego}}(w) \log \frac{p_{\text{stego}}(w)}{p_{\text{baseline}}(w)}$; lower KL indicates closer statistical match to the baseline. KL being lower against the global corpus than against the matched post suggests stego wording drifts toward generic phrasing rather than thread-specific vocabulary.

Jensen–Shannon divergence (JSD) is the symmetric, bounded counterpart, $D_{\text{JS}}(P_C \| P_S) = \frac{1}{2} D_{\text{KL}}(P_C \| \frac{P_C + P_S}{2}) + \frac{1}{2} D_{\text{KL}}(P_S \| \frac{P_C + P_S}{2})$, often used to upper-bound distinguishability from the true language distribution. Word pairs absent from one side of the comparison are handled with Laplace-style smoothing (e.g., $\alpha = 10^{-6}$) so KL and JSD remain well defined.

5.3 System Robustness & Recovery Rate

The sender-side verification loop improves operational reliability by rejecting candidates that do not decode back to the selected Angle. In a representative clean run (requested=200), the pipeline produced 183 successful, receiver-validated samples and 17 failures after exhausting the retry budget. The primary failure modes were (i) decoding validation failures (10/17) and (ii) malformed synthesis output that did not match the required JSON schema (7/17). These failures highlight that reliability is primarily limited by the ability to synthesize a decodable witness under stochastic generation, rather than by payload recovery after validation.

5.4 Resistance to Automated Steganalysis

The design objective of the framework is to reduce semantic and pragmatic anomalies by conditioning synthesis on the full thread context and by encoding information in conversational decisions (reply targeting and Angle selection) rather than local token artifacts. We therefore expect detection to require thread-level pragmatic reasoning rather than relying solely on token-level distribution cues [4]. A dedicated steganalysis benchmark evaluation is left to future work; here we limit claims to the measured distributional and operational metrics reported above.

6 CONCLUSION

This research introduces a context-aware semantic steganography framework for threaded discussions that redefines the embedding domain as a *selection channel*. By transitioning from token-level manipulation to encoding payload bits into reply targeting and Angle selection, the method aims to preserve contextual naturalness in the visible text while retaining black-box compatibility with LLM APIs.

The implementation includes a closed-loop verification mechanism that simulates decoding prior to publication and retries synthesis under stochastic generation. Our artifact-based evaluation reports naturalness and unigram divergence metrics on successful, receiver-validated samples, and characterizes the dominant failure modes when retries are exhausted. Future work will focus on improving end-to-end success rates on fresh real threads, expanding clean sample sizes, and performing dedicated steganalysis evaluations under explicit adversarial models.

REFERENCES

- [1] Jipeng Qiang, Shiyu Zhu, Yun Li, Yi Zhu, Yunhao Yuan, and Xindong Wu. Natural language watermarking via paraphraser-based lexical substitution. *Artificial Intelligence*, 317:103859, 2023.
- [2] Zhong-Liang Yang, Si-Yu Zhang, Yu-Ting Hu, Zhi-Wen Hu, and Yong-Feng Huang. Vae-stega: linguistic steganography based on variational auto-encoder. *IEEE Transactions on Information Forensics and Security*, 16:880–895, 2020.

- [3] Gabriel Kaptchuk, Tushar M Jois, Matthew Green, and Aviel D Rubin. Meteor: Cryptographically secure steganography for realistic distributions. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 1529–1548, Virtual Event, Republic of Korea, 2021. ACM.
- [4] Yihao Wang, Ru Zhang, Yifan Tang, and Jianyi Liu. State-of-the-art advances of deep-learning linguistic steganalysis research. In *2023 International Conference on Data, Information and Computing Science (CDICS)*, page 20–24. IEEE, December 2023. doi: 10.1109/cdics61497.2023.00014. URL <http://dx.doi.org/10.1109/CDICS61497.2023.00014>.
- [5] Fanxiao Li, Sixing Wu, Jiong Yu, Shuoxin Wang, BingBing Song, Renyang Liu, Haoseng Lai, and Wei Zhou. Rewriting-stego: generating natural and controllable steganographic text with pre-trained language model. In *International Conference on Database Systems for Advanced Applications*, pages 617–626. Springer, 2023.
- [6] Ke Lin, Yiyang Luo, Zijian Zhang, and Ping Luo. Zero-shot generative linguistic steganography. *arXiv preprint arXiv:2403.10856*, 2024.
- [7] Jiaxuan Wu, Zhengxian Wu, Yiming Xue, Juan Wen, and Wanli Peng. Generative text steganography with large language model. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 10345–10353, Melbourne, Australia, 2024. ACM.
- [8] Guorui Liao, Jinshuai Yang, Kaiyi Pang, and Yongfeng Huang. Co-stega: Collaborative linguistic steganography for the low capacity challenge in social media. In *Proceedings of the 2024 ACM Workshop on Information Hiding and Multimedia Security*, pages 7–12, Baiona, Spain, 2024. ACM.
- [9] Huili Wang, Zhongliang Yang, Jinshuai Yang, Yue Gao, and Yongfeng Huang. Hi-stega: A hierarchical linguistic steganography framework combining retrieval and generation. In *International Conference on Neural Information Processing*, pages 41–54. Springer, 2023.
- [10] Ruifan Zhang, Jianyi Liu, and Ru Zhang. Controllable semantic linguistic steganography via summarization generation. In *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4560–4564. IEEE, 2024.
- [11] Gustavus J Simmons. The prisoners’ problem and the subliminal channel. In *Advances in Cryptology: Proceedings of Crypto 83*, pages 51–67, Boston, MA, 1984. Springer.
- [12] Jessica Fridrich. *Steganography in digital media: principles, algorithms, and applications*. Cambridge University Press, Cambridge, UK, 2009.
- [13] Murray Shanahan. Talking about large language models. *Communications of the ACM*, 67(2):68–79, 2024.
- [14] Changhao Ding, Zhangjie Fu, Zhongliang Yang, Qi Yu, Daqiu Li, and Yongfeng Huang. Context-aware linguistic steganography model based on neural machine translation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 32:868–878, 2023.
- [15] Yequan Wang, Jiawen Deng, Aixin Sun, and Xuying Meng. Perplexity from plm is unreliable for evaluating text quality. *arXiv preprint arXiv:2210.05892*, 2022.